CS1231S Discrete Structures

WEEK 12 RECITATION
LECTURE 13. TREES

5 – 6 November 2026

Aaron Tan



CS1231S (AY2026/27 Sem1) QnA



Trees



Definition: Tree

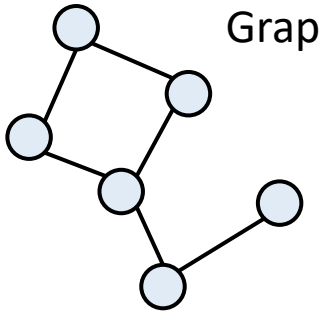
(The graph is assumed to be undirected here.)

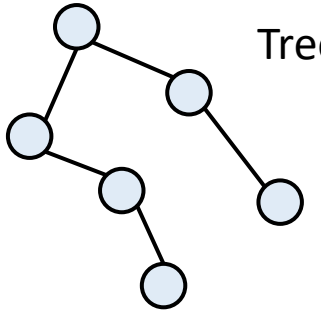
A **graph** is said to be **circuit-free** if and only if it has no circuits.

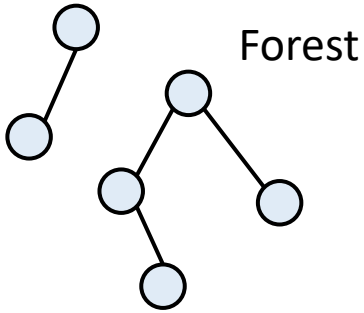
A simple graph is called a **tree** if and only if it is circuit-free and connected.

A **trivial tree** is a tree that consists of a single vertex.

A simple graph is called a **forest** if and only if it is circuit-free and not connected.



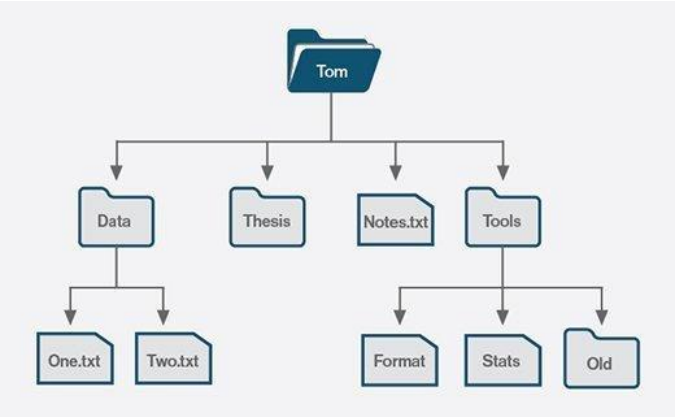




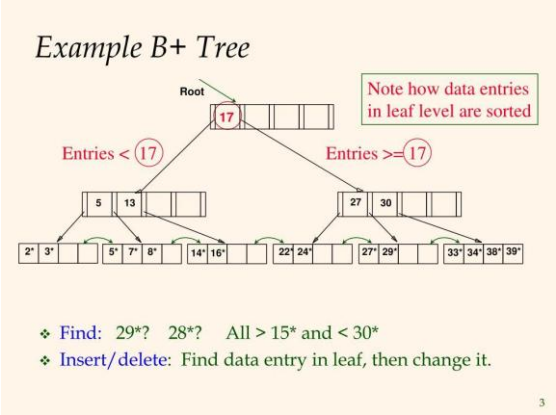
Trees: Applications

Trees are used to store hierarchically ordered data.

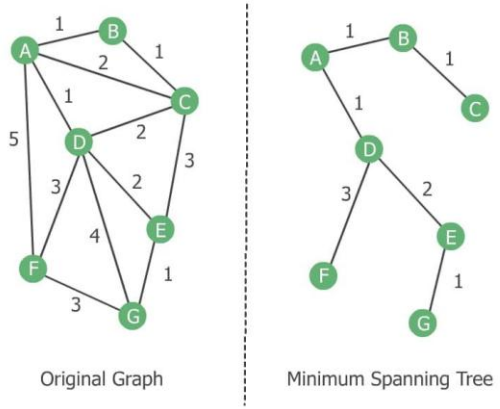
File systems



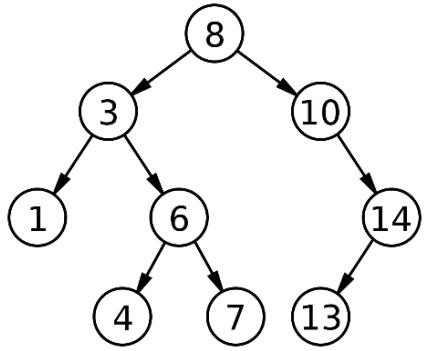
Database indexing



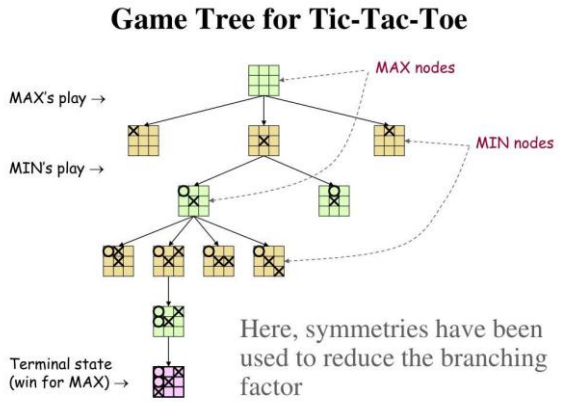
Network design



Searching and sorting



Game playing and AI search



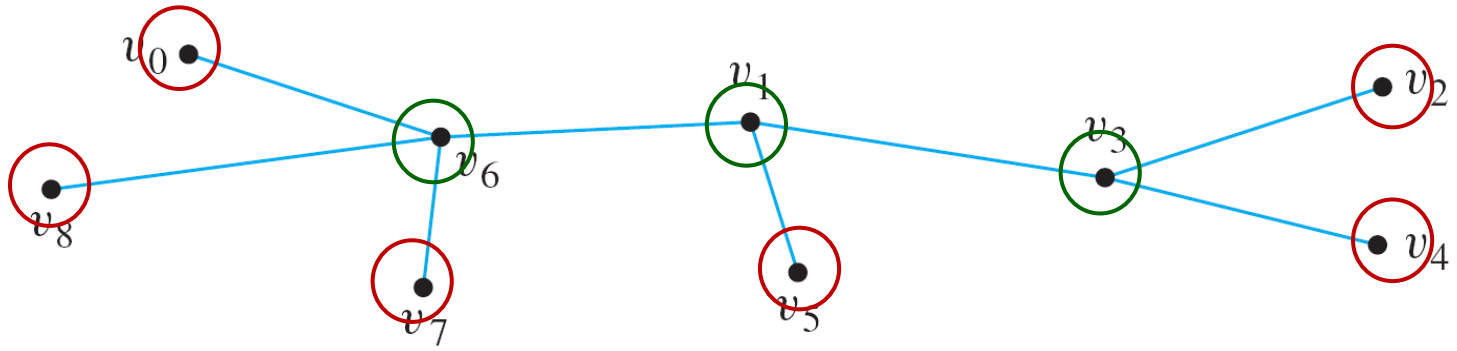
And many more…

Trees: Definitions

Definitions: Terminal vertex (leaf) and internal vertex

Let T be a tree. If T has only one or two vertices, then each is called a **terminal vertex** (or **leaf**). If T has at least three vertices, then a vertex of degree 1 in T is called a **terminal vertex** (or **leaf**), and a vertex of degree greater than 1 in T is called an **internal vertex**.

Example: Find all terminal vertices (leaves) and all internal vertices in the following tree:



Leaves: v_0, v_2, v_4, v_5, v_7 and v_8.

Internal vertices: v_6, v_1 and v_3.

Lemma 10.5.1

Any non-trivial tree has at least one vertex of degree 1.

Proof: Let T be a particular but arbitrarily chosen non-trivial tree.

Step 1: Pick a vertex v of T and let e be an edge incident on v.

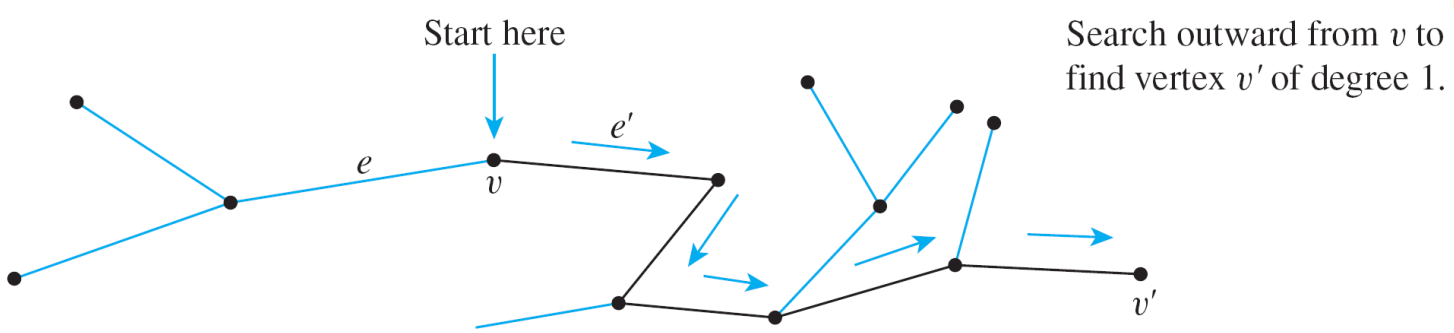
Step 2: While $\deg(v) > 1$, repeat steps 2a, 2b and 2c:

2a: Choose e' to be an edge incident on v such that $e' \neq e$.

2b: Let v' be the vertex at the other end of e' from v. Since T is acyclic, v' cannot be a previously visited vertex.

2c: Let $e = e'$ *and* $v = v'$.

The algorithm must eventually terminate because the set of vertices of the tree T is finite and T is circuit-free. When it does, a vertex v of degree 1 will have been found.



Theorem 10.5.2

Any tree with n vertices ($n > 0$) has $n - 1$ edges.

Proof: By mathematical induction.

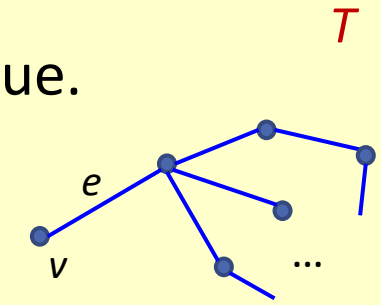
Let the property $P(n)$ be "any tree with n vertices has $n - 1$ edges".

$P(1)$: Let T be a tree with one vertex. Then T has no edges.
So $P(1)$ is true.

To show that T has k edges.

Show that for arbitrary integer $k \geq 1$, if $P(k)$ is true then $P(k+1)$ is true.

Suppose $P(k)$ is true.



1. Let T be a particular but arbitrarily chosen tree with $k + 1$ vertices.
2. Since k is positive, $(k + 1) \geq 2$, and so T has more than one vertex.
3. Hence, by Lemma 10.5.1, T has a vertex v of degree 1, and has at least another vertex in T besides v.

Theorem 10.5.2

Any tree with n vertices ($n > 0$) has $n - 1$ edges.

Proof: (continued…)

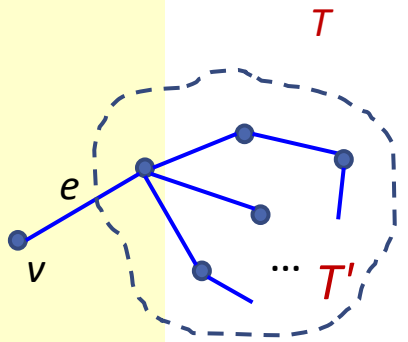
3. Hence, by Lemma 10.5.1, T has a vertex v of degree 1, and has at least another vertex in T besides v.
4. Thus, there is an edge e connecting v to the rest of T.
5. Define a subgraph T' of T so that $V_{T'} = V_T - \{v\}$ and $E_{T'} = E_T - \{e\}$.

   5.1 The number of vertices of T' is $(k + 1) - 1 = k$.

   5.2 T' is circuit-free.

   5.3 T' is connected.
6. Hence by definition, T' is a tree.
7. Since T' has k vertices, by inductive hypothesis, number of edges of T' = (number of vertices of T') $- 1 = k - 1$.
8. Hence, number of edges of T = (number of edges of T') $+ 1 = k$.
9. Hence $P(k+1)$ is true.

To show that T has k edges.



Trees

Lemma 10.5.1

Any non-trivial tree has at least one vertex of degree 1.

Theorem 10.5.2

Any tree with n vertices ($n > 0$) has $n - 1$ edges.

Exercise: Using Theorem 10.5.2, prove that a non-trivial tree has at least 2 vertices of degree 1.

1. Let $T = (V, E)$ be a particular but arbitrary chosen non-trivial tree with n vertices, $n \geq 2$.
2. By Lemma 10.5.1, T has a vertex of degree 1.
3. Suppose it has exactly one such vertex.
   - 3.1. Since T is connected, every other vertex has degree at least 2.
   - 3.2. Hence $\sum_{v \in V} \deg(v) \geq 1 + 2(n-1) = 2n - 1$.
   - 3.3. But T has $n - 1$ edges (Theorem 10.5.5), so by the Handshake Theorem, $\sum_{v \in V} \deg(v) = 2(n-1) = 2n - 2$, hence contradiction.
4. Therefore, T has at least two vertices of degree 1.

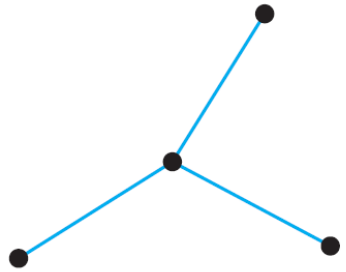
Theorem 10.1.1 The Handshake Theorem

Total degree of a graph $G = (V, E)$ is $2|E|$.

Trees

Example: Find all non-isomorphic trees with 4 vertices.

1. By Theorem 10.5.2, any tree with 4 vertices has 3 edges.
2. By the Handshake Theorem, the tree has a total degree of 6.
3. By the previous exercise, every non-trivial tree has at least 2 vertices of degree 1.
4. The only possible combinations of degrees are: 1,1,1,3 and 1,1,2,2.
5. Therefore, there are 2 non-isomorphic trees with 4 vertices.



Every non-trivial tree has at least 2 vertices of degree 1.

Theorem 10.5.2

Any tree with n vertices ($n > 0$) has $n - 1$ edges.

Theorem 10.1.1 The Handshake Theorem

Total degree of a graph $G = (V, E)$ is $2|E|$.

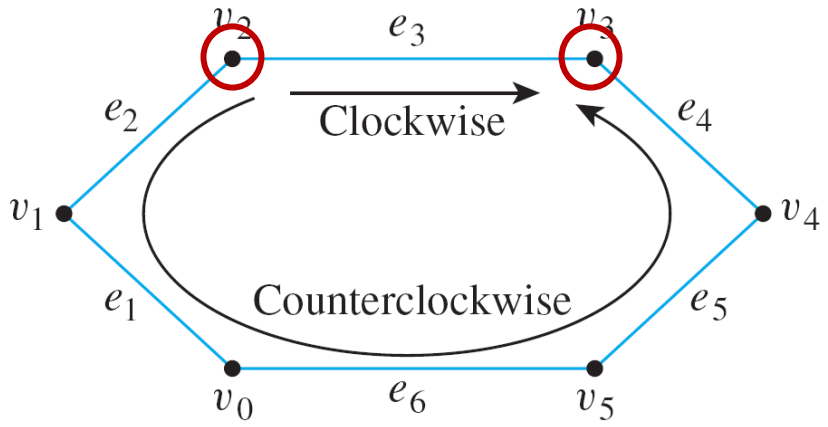
Trees

Lemma 10.5.3

If G is any connected graph, C is any circuit in G, and one of the edges of C is removed from G, then the graph that remains is still connected.

Any two vertices in a circuit are connected by 2 distinct paths. It is possible to draw the graph so that one of these goes "clockwise" and the other goes "counter-clockwise" around the circuit.

For example, in the circuit shown below:



The clockwise path from v_2 to v_3 is: $v_2\ e_3\ v_3$

The counter-clockwise path from v_2 to v_3 is:

$v_2\ e_2\ v_1\ e_1\ v_0\ e_6\ v_5\ e_5\ v_4\ e_4\ v_3$

Trees

Theorem 10.5.4

If G is a connected graph with n vertices and $n - 1$ edges, then G is a tree.

Proof:

1. Suppose G is a particular but arbitrarily chosen graph that is connected and has n vertices and $n - 1$ edges.
2. Since G is connected, it suffices to show that G is circuit-free.
3. Suppose G is not circuit free
   - 3.1 Let C be the circuit in G.
   - 3.2 By Lemma 10.5.3, an edge of C can be removed from G to obtain a graph G' that is connected.
   - 3.3 If G' has a circuit, then repeat this process: Remove an edge of the circuit from G' to form a new connected graph.
   - 3.4 Continue the process of removing edges from the circuits until eventually a graph G'' is obtained that is connected and is circuit-free.

Trees

Theorem 10.5.4

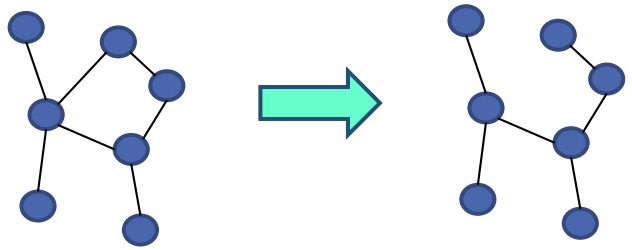
If G is a connected graph with n vertices and $n - 1$ edges, then G is a tree.

Proof: (continued…)

3.4 Continue the process of removing edges from the circuits until eventually a graph G'' is obtained that is connected and is circuit-free.
3.5 By definition, G'' is a tree.
3.6 Since no vertices were removed from G to form G'', G'' has n vertices.
3.7 Thus, by Theorem 10.5.2, G'' has $n - 1$ edges.
3.8 But the supposition that G has a circuit implies that at least one edge of G is removed to form G''.
3.9 Hence G'' has no more than $(n - 1) - 1 = n - 2$ edges, which contradicts its having $n - 1$ edges.
3.10 So the supposition is false.

4. Hence G is circuit-free, and therefore G is a tree.

Assume G is not circuit-free.
G has n vertices and $n - 1$ edges.



G'' is the result of removing edges from circuits in G. At least 1 edge removed from G. G'' has n vertices and at most $n - 2$ edges.

Rooted Trees

Definitions: Rooted Tree, Level, Height

A **rooted tree** is a tree in which there is one vertex that is distinguished from the others and is called the **root**.

The **level** of a vertex is the number of edges along the unique path between it and the root.

The **height** of a rooted tree is the maximum level of any vertex of the tree.

Definitions: Child, Parent, Sibling, Ancestor, Descendant

Given the root or any internal vertex v of a rooted tree, the **children** of v are all those vertices that are adjacent to v and are one level farther away from the root than v.

If w is a child of v, then v is called the **parent** of w, and two distinct vertices that are both children of the same parent are called **siblings**.

Given two distinct vertices v and w, if v lies on the unique path between w and the root, then v is an **ancestor** of w, and w is a **descendant** of v.

Rooted Trees

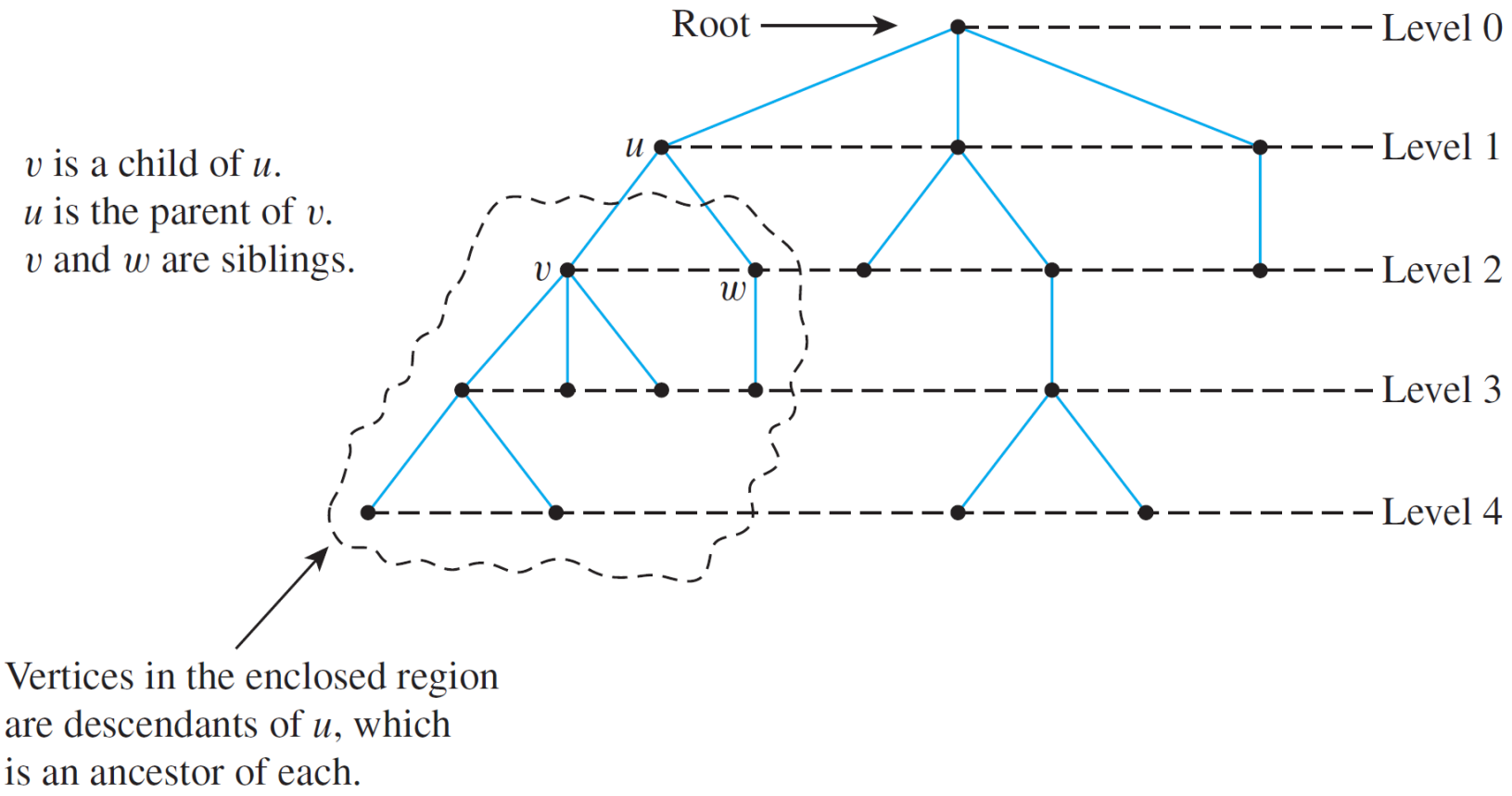


Figure 10.6.1 A Rooted Tree

Binary Trees, Full Binary Trees

Definitions: Binary Tree, Full Binary Tree

A **binary tree** is a rooted tree in which every parent has at most two children. Each child is designated either a **left child** or a **right child** (but not both), and every parent has at most one left child and one right child.

A **full binary tree** is a binary tree in which each parent has exactly two children.

Definitions: Left Subtree, Right Subtree

Given any parent v in a binary tree T, if v has a left child, then the **left subtree** of v is the binary tree whose root is the left child of v, whose vertices consist of the left child of v and all its descendants, and whose edges consist of all those edges of T that connect the vertices of the left subtree.

The **right subtree** of v is defined analogously.

Binary Trees

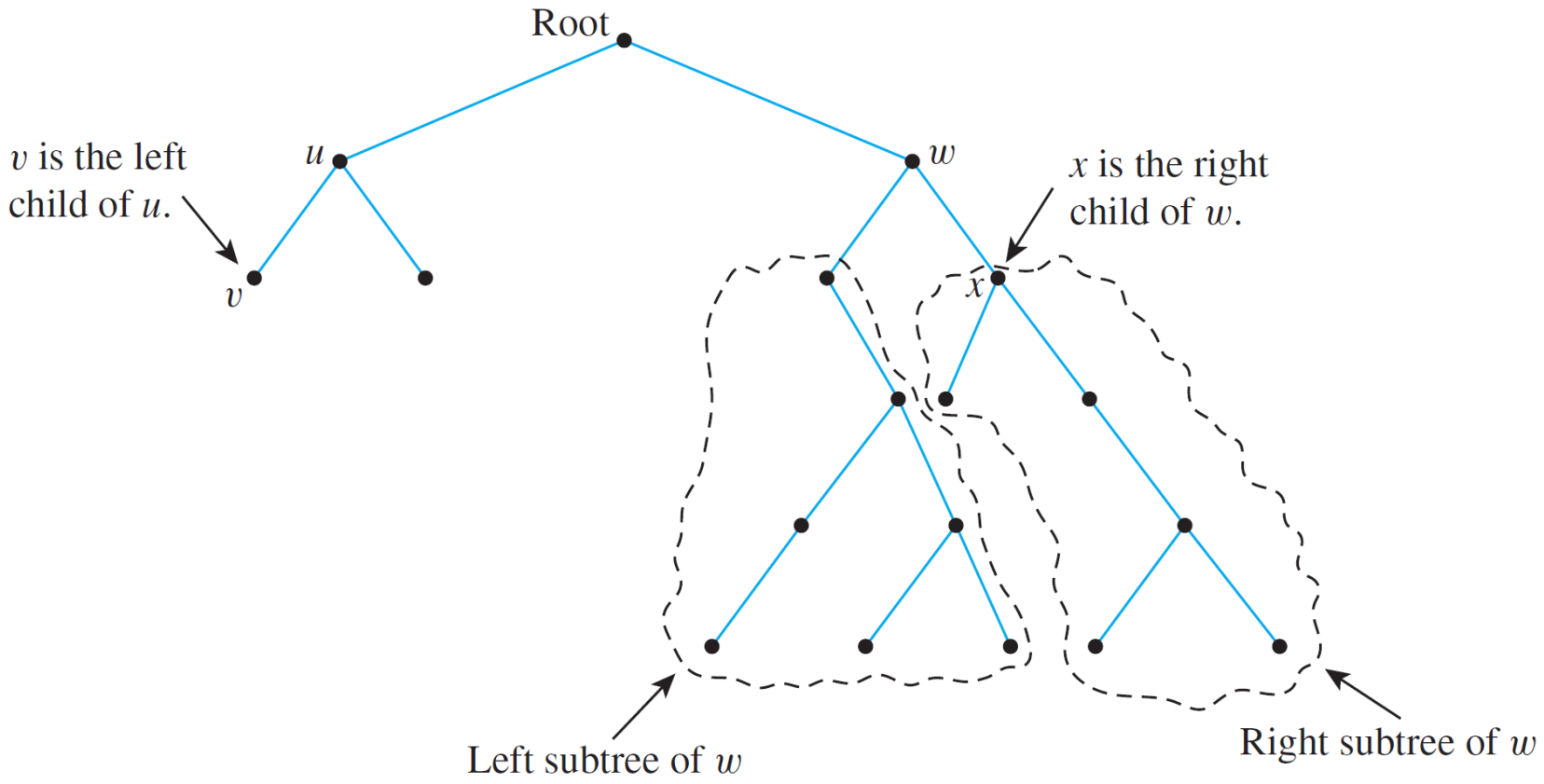


Figure 10.6.2 A Binary Tree

Full Binary Trees

Theorem 10.6.1: Full Binary Tree Theorem

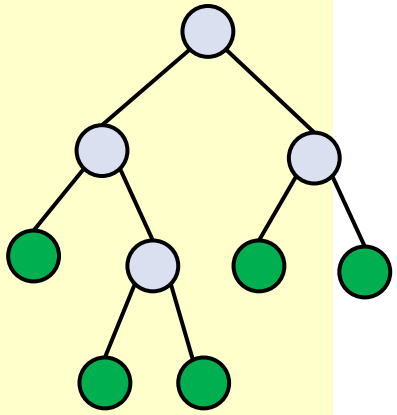
If T is a full binary tree with k internal vertices, then T has a total of $2k + 1$ vertices and has $k + 1$ terminal vertices (leaves).

Proof:

1. Every vertex, except the root, has a parent.
2. Since every internal vertex of a full binary tree has exactly two children, the number of vertices that have a parent is twice the number of parents, or $2k$.

   #vertices of T = #vertices that have a parent + #vertices that do not have a parent

   $= 2k + 1$



3. #terminal vertices = #vertices – #internal vertices

   $= 2k + 1 - k = k + 1$
4. Therefore, T has a total of **$2k + 1$ vertices** and has **$k + 1$ terminal vertices**.

Binary Trees

Theorem 10.6.2

If T is any binary tree with height h ($h \geq 0$) and t terminal vertices (leaves), then $t \leq 2^h$. Equivalently, $\log_2 t \leq h$.

Proof: By strong mathematical induction

1. Let $P(h)$ be "If T is any binary tree of height h, then the number of leaves of T is at most 2^h."
2. $P(0)$: T consists of one vertex, which is a terminal vertex. Hence $t = 1 = 2^0$.
3. Show that for all integers $k \geq 0$, if $P(i)$ is true for all integers i from 0 through k, then $P(k+1)$ is true.
4. Let T be an arbitrary binary tree of height $k + 1$, root v, and t leaves.
5. Since $k \geq 0$, hence $k + 1 \geq 1$ and so v has at least one child.
6. We consider two cases: If v has only one child, or if v has two children.

Binary Trees

Theorem 10.6.2

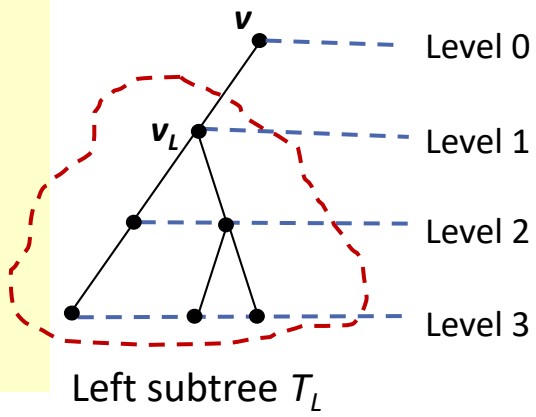
If T is any binary tree with height h ($h \geq 0$) and t terminal vertices (leaves), then $t \leq 2^h$. Equivalently, $\log_2 t \leq h$.

Proof: (continued…)

P(k+1): T of height k+1 has $t \leq 2^{k+1}$.

7. Case 1 (v has only one child):
   - 7.1 Without loss of generality, assume that v's child is a left child and denote it by v_L. Let T_L be the left subtree of v.
   - 7.2 Because v has only one child, v has degree 1 (leaf), so the total number of leaves in T equals the number of leaves in T_L + 1. Thus, if t_L is the number of leaves in T_L , then $t = t_L + 1$.
   - 7.3 By inductive hypothesis, $t_L \leq 2^k$ because the height of T_L is k, one less than the height of T.
   - 7.4 Also, because v has a child, $k+1 \geq 1$ and so $2^k \geq 2^0 = 1$.
   - 7.5 Therefore,

$$t = t_L + 1 \leq 2^k + 1 \leq 2^k + 2^k = 2^{k+1}$$



Binary Trees

Theorem 10.6.2

If T is any binary tree with height h ($h \geq 0$) and t terminal vertices (leaves), then $t \leq 2^h$. Equivalently, $\log_2 t \leq h$.

Proof: (continued…)

8. Case 2 (v has two children):

P(k+1): T of height k+1 has $t \leq 2^{k+1}$.

   8.1 Now v has a left child v_L and a right child v_R , and they are the roots of a left subtree T_L and a right subtree T_R respectively.

   8.2 Let h_L and h_R be the heights of T_L and T_R respectively.

   8.3 Then $h_L \leq k$ and $h_R \leq k$ since T is obtained by joining T_L and T_R and adding a level.

   8.4 Let t_L and t_R be the number of leaves of T_L and T_R respectively.

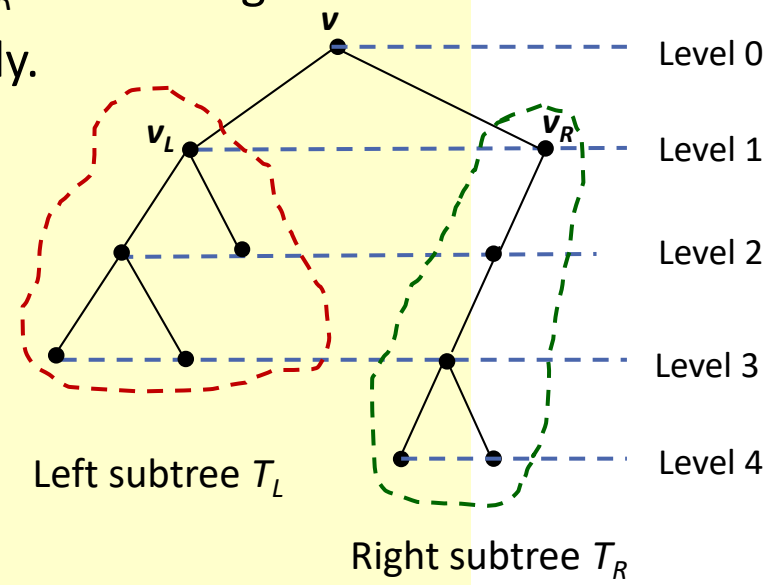
   8.5 Then, since both T_L and T_R have heights less than $k + 1$, by inductive hypothesis, $t_L \leq 2^{h_L}$ and $t_R \leq 2^{h_R}$.

   8.6 Therefore,

$$t = t_L + t_R \leq 2^{h_L} + 2^{h_R} \leq 2^k + 2^k = 2^{k+1}$$

9. In both cases, $P(k+1)$ is true.

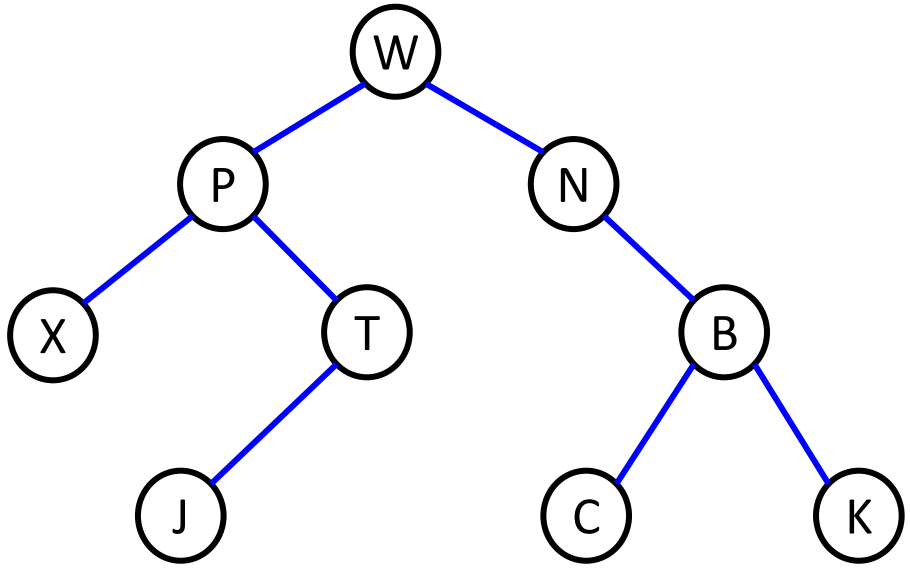
10. Hence if T is any binary tree with height h and t terminal vertices (leaves), then $t \leq 2^h$.



Binary Tree Traversal: Breadth-First Search (BFS)

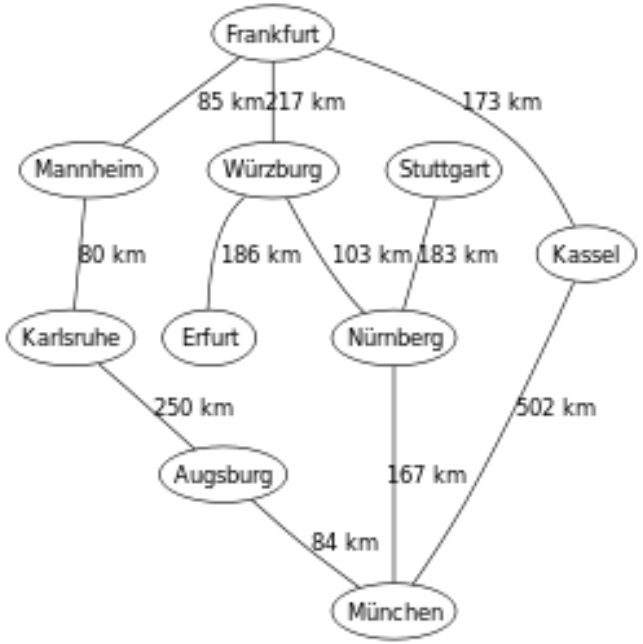
In breadth-first search (by E.F. Moore), it starts at the root and visits its adjacent vertices, and then moves to the next level.

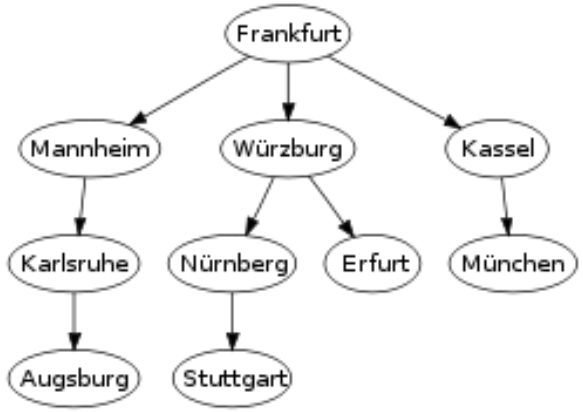
BFS: W P N X T B J C K



Binary Tree Traversal: Breadth-First Search (BFS)

The figure on the left shows a graph representing cities in Germany. The figure on the right shows the breadth-first traversal on the graph, starting with Frankfurt.





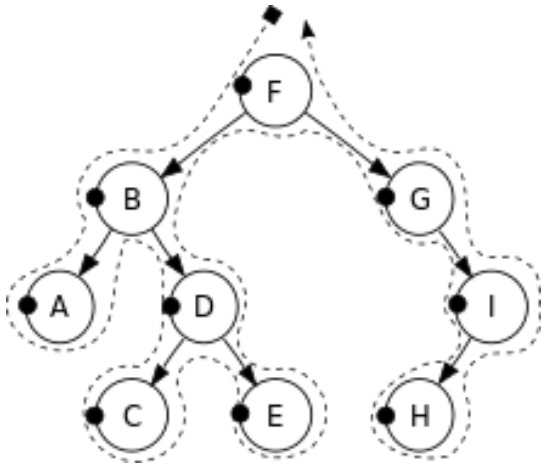
Acknowledgement: Wikipedia https://en.wikipedia.org/wiki/Breadth-first_search

Binary Tree Traversal: Depth-First Search (DFS)

There are 3 types of depth-first traversal:

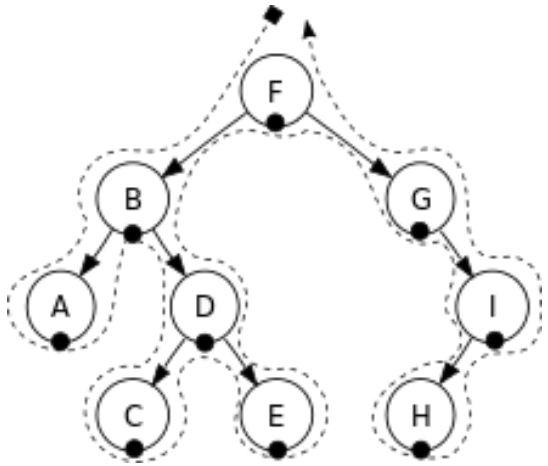
- Pre-order
  - Print the data of the root (or current vertex)
  - Traverse the left subtree by recursively calling the pre-order function
  - Traverse the right subtree by recursively calling the pre-order function
- In-order
  - Traverse the left subtree by recursively calling the in-order function
  - Print the data of the root (or current vertex)
  - Traverse the right subtree by recursively calling the in-order function
- Post-order
  - Traverse the left subtree by recursively calling the post-order function
  - Traverse the right subtree by recursively calling the post-order function
  - Print the data of the root (or current vertex)

Binary Tree Traversal: Depth-First Search (DFS)



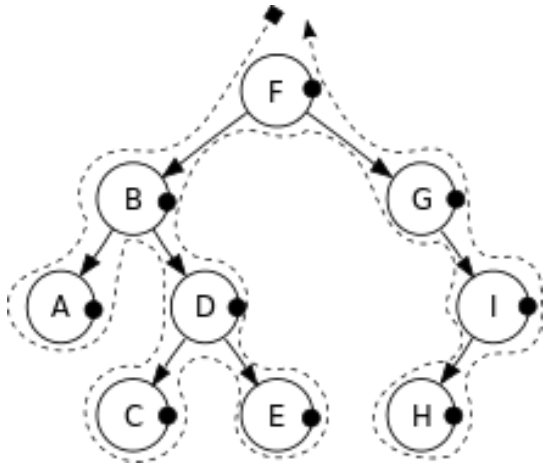
Pre-order:

F, B, A, D, C, E, G, I, H



In-order:

A, B, C, D, E, F, G, H, I



Post-order:

A, C, E, D, B, H, I, G, F

Acknowledgement: Wikipedia https://en.wikipedia.org/wiki/Tree_traversal

Past-Year's Exam Question!

AY2018/19 Semester 1 Q17

The lazy caterer's sequence describes the number of maximum pieces of a pancake (or pizza) that can be made with a given number of straight cuts.

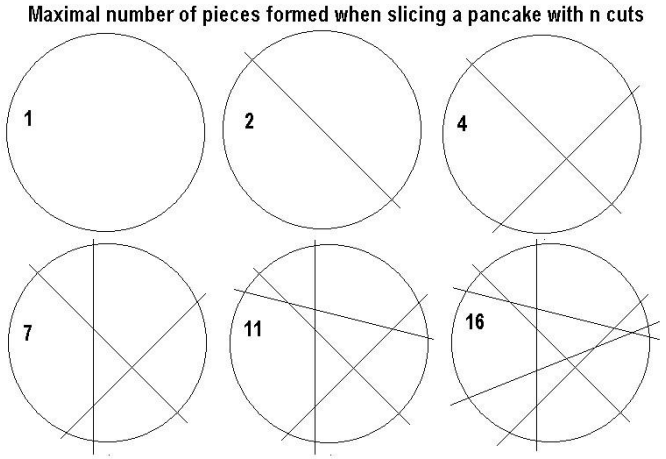
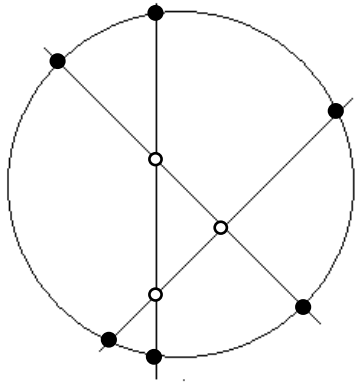


Figure 1: Lazy Caterer's Sequence.



$P(3) = 7;$
$V(3) = 9;$
$V_3(3) = 6;$
$V_4(3) = 3;$
$E(3) = 15.$

Figure 2: Graph representation ($n = 3$).

$P(n)$: number of pieces of pancakes with n cuts;

$V(n)$: number of vertices of a graph corresponding to a pancake with n cuts;

- $V_3(n)$: number of vertices with degree 3;
- $V_4(n)$: number of vertices with degree 4;
- $V(n) = V_3(n) + V_4(n)$.

$E(n)$: number of edges of a graph corresponding to a pancake with n cuts.

Past-Year's Exam Question!

AY2018/19 Semester 1 Q17

The lazy caterer's sequence describes the number of maximum pieces of a pancake (or pizza) that can be made with a given number of straight cuts.

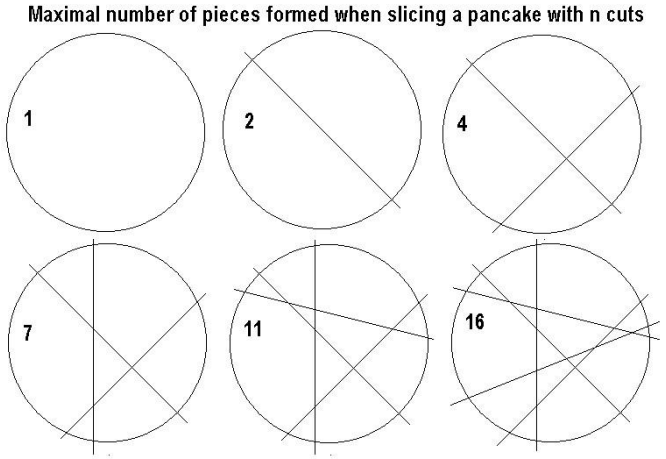
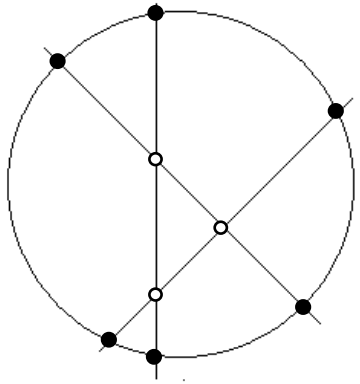


Figure 1: Lazy Caterer's Sequence.



$P(3) = 7;$
$V(3) = 9;$
$V_3(3) = 6;$
$V_4(3) = 3;$
$E(3) = 15.$

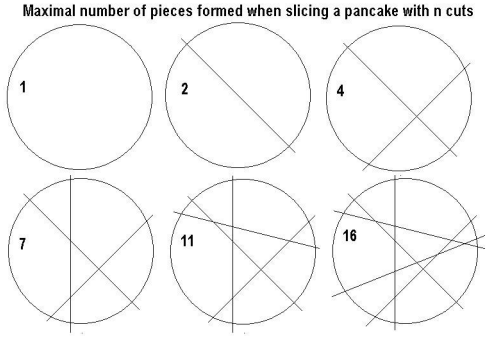
Figure 2: Graph representation ($n = 3$).

(a) Express $E(n)$ in terms of $V_3(n)$ and $V_4(n)$.
(b) Write the recurrence relation for $V(n)$. The base case is $V(0) = 0$.
(c) Write the closed form formula for $V(n)$.
(d) Write the recurrence relation for $E(n)$. The base case is $E(0) = 0$.
(e) Write the closed form formula for $E(n)$.
(f) Euler's formula is given as $v - e + f = 2$. Relate v, e and f with the functions defined in this question.
(g) From part (f), or otherwise, derive the closed form formula for $P(n)$.

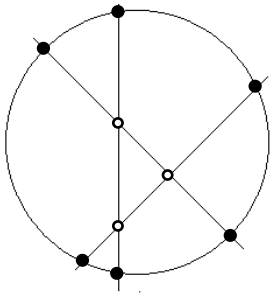
Past-Year's Exam Question!

AY2018/19 Semester 1 Q17

The lazy caterer's sequence describes the number of maximum pieces of a pancake (or pizza) that can be made with a given number of straight cuts.



$n = 3$



n	0	1	2	3	4	5	6	7
$P(n)$	1	2	4	7	11	16	22	29
$E(n)$	0	3	8	15	24	35	48	63
$V(n)$	0	2	5	9	14	20	27	35
$V_3(n)$	0	2	4	6	8	10	12	14
$V_4(n)$	0	0	1	3	6	10	15	21

$V_3(n) = 2n$ (every cut creates 2 new vertices on the boundary)

$V_4(n) = V_4(n-1) + n - 1$ (the nth cut intersects the existing $n-1$ cuts)

Or, $V_4(n) = n(n-1)/2$ for $n > 0$.

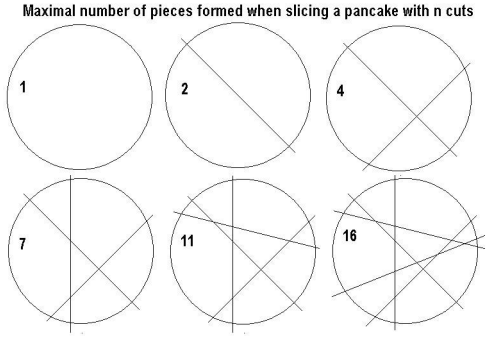
$V(n) = V(n-1) + n + 1$ (the nth cut cuts through the existing $n-1$ cuts and it also cuts the boundary at two places)

Or, $V(n) = V_3(n) + V_4(n) = 2n + \frac{n(n-1)}{2} = \frac{n^2+3n}{2}$.

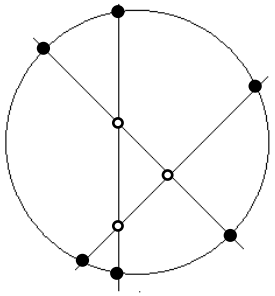
Past-Year's Exam Question!

AY2018/19 Semester 1 Q17

The lazy caterer's sequence describes the number of maximum pieces of a pancake (or pizza) that can be made with a given number of straight cuts.



$n = 3$



n	0	1	2	3	4	5	6	7
$P(n)$	1	2	4	7	11	16	22	29
$E(n)$	0	3	8	15	24	35	48	63
$V(n)$	0	2	5	9	14	20	27	35
$V_3(n)$	0	2	4	6	8	10	12	14
$V_4(n)$	0	0	1	3	6	10	15	21

$E(n) = E(n-1) + 2n + 1$ (the nth cut adds $n+1$ vertices as it cuts through the existing $n-1$ cuts and opposite sides of the circle, splitting each of the $n+1$ edges it cuts through into 2, and also introducing n new edges).

$E(n) = \frac{3V_3(n)+4V_4(n)}{2}$ (Handshake Theorem: total degree of a graph is twice the number of edges)

$= \frac{3(2n)+2n(n-1)}{2} = n^2 + 2n.$

From previous slide:

$V_3(n) = 2n$

$V_4(n) = \frac{n(n-1)}{2}$

$P(n) = E(n) - V(n) + 1$ (using Euler's formula $v - e + f = 2$, where $f = P(n) + 1$ since the region outside the circle constitutes one face)

Or, $P(n) = P(n-1) + n.$

All the above give the closed form formula $P(n) = \frac{n^2+n+2}{2}$.

Past-Year's Exam Question!

AY2018/19 Semester 1 Q17

The lazy caterer's sequence describes the number of maximum pieces of a pancake (or pizza) that can be made with a given number of straight cuts.

(a) Express $E(n)$ in terms of $V_3(n)$ and $V_4(n)$. $\boldsymbol{E(n) = \frac{3V_3(n) + 4V_4(n)}{2}}$.

(b) Write the recurrence relation for $V(n)$. The base case is $V(0) = 0$.
$\boldsymbol{V(n) = V(n-1) + n + 1, \text{ for } n > 0.}$

(c) Write the closed form formula for $V(n)$. $\boldsymbol{V(n) = \frac{n^2+3n}{2}, \text{ for } n \geq 0.}$

(d) Write the recurrence relation for $E(n)$. The base case is $E(0) = 0$.
$\boldsymbol{E(n) = E(n-1) + 2n + 1, \text{ for } n > 0.}$

(e) Write the closed form formula for $E(n)$. $\boldsymbol{E(n) = n^2 + 2n, \text{ for } n \geq 0.}$

(f) Euler's formula is given as $v - e + f = 2$. Relate v, e and f with the functions defined in this question. $\boldsymbol{v = V(n), e = E(n), \text{ and } f = P(n) + 1.}$

(g) From part (f), or otherwise, derive the closed form forum for $P(n)$. $\boldsymbol{P(n) = \frac{n^2+n+2}{2}, \text{ for } n \geq 0.}$

Using AI

Dos and Don'ts of Using AI in ~~CS1231S~~ Any Course

	Example
Attempt the problem yourself first.	Work on an induction proof before consulting AI.
Ask for hints rather than complete solutions.	"Give me a hint for proving this statement by contradiction, but don't reveal the proof."
Ask AI to explain difficult concepts.	"Explain the difference between countable and uncountable sets using examples."
Ask AI to critique your reasoning.	"Here's my proof. Identify any logical gaps without rewriting it for me."
Ask for alternative approaches.	"Can this problem be solved using both direct proof and contrapositive?"
Use AI to generate practice problems.	"Give me three problems on graph isomorphism, with increasing difficulty."
Challenge AI's answers.	"Are you sure this graph is bipartite? Can you justify your answer?"
Use AI to deepen your understanding.	"Why does proving the inductive step alone not establish that a statement holds for all natural numbers?"

Dos and Don'ts of Using AI in ~~CS1231S~~ Any Course

	Why
Don't ask AI to solve a problem before attempting it.	You lose the opportunity to develop problem-solving skills.
Don't blindly copy AI-generated proofs.	A correct-looking proof may contain subtle logical errors.
Don't assume AI is always correct.	AI can produce invalid arguments, incorrect calculations, and false claims.
Don't confuse understanding a solution with being able to produce one.	Reading a proof is easier than constructing it independently
Don't rely on AI to do all the mathematical reasoning.	Your reasoning skills improve through practice, not observation alone.
Don't use AI to bypass assignment requirements..	Follow the course's rules on permitted AI assistance and academic integrity.
Don't accept an answer just because it sounds convincing.	Mathematical correctness requires rigorous justification.
Don't become dependent on AI.	You should eventually be able to solve similar problems without assistance.

Using AI: A 5-step Approach

1. Think first

Attempt the problem independently. Write down your ideas, even if they are incomplete.

2. Ask for guidance

If stuck, request a hint, a relevant theorem, or an explanation of the underlying concept.

3. Solve the problem yourself

Continue working using the guidance, without simply copying an AI-generated solution.

4. Verify and critique

Ask AI to examine your solution for errors. Independently check any feedback it provides.

5. Reflect and reproduce

Put AI aside and solve the problem again. Explain why each step works and what you have learned.

Spanning Trees

An East Coast airline company wants to expand service to the Midwest and has received permission from the Federal Aviation Authority to fly any of the routes shown in Figure 10.7.1.

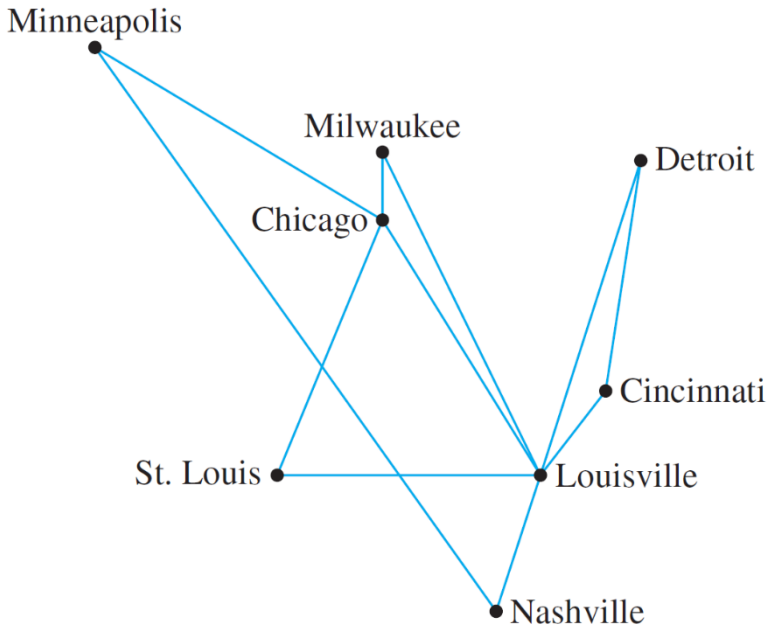


Figure 10.7.1

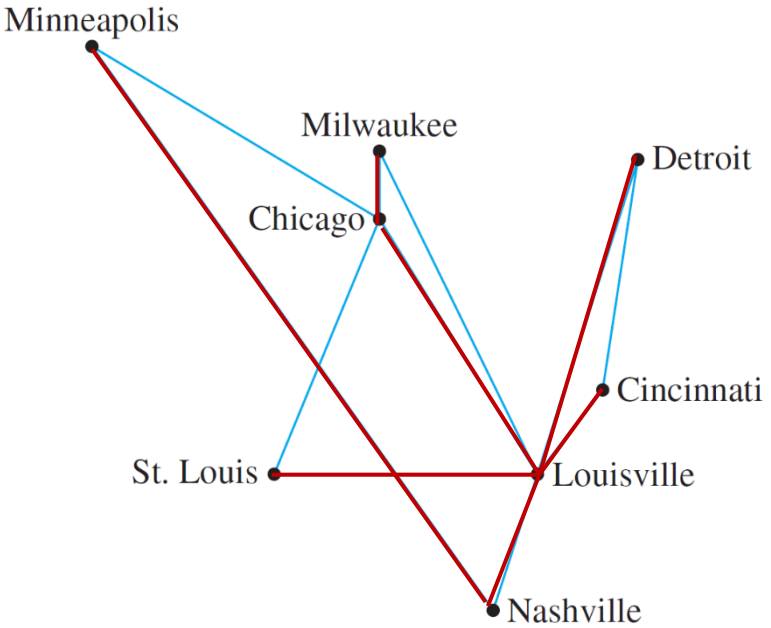


Figure 10.7.2

The company wishes to legitimately advertise service to all the cities shown but, for reasons of economy, wants to use the least possible number of individual routes to connect them. One possible route system is given in Figure 10.7.2, where the chosen routes are in red.

Spanning Trees

Is the number of individual routes minimal?

The fact is that the graph of any system of routes that satisfies the company's wishes is a tree, because if the graph were to contain a circuit, then one of the routes in the circuit could be removed without disconnecting the graph (by Lemma 10.5.3), and that would give a smaller total number of routes.

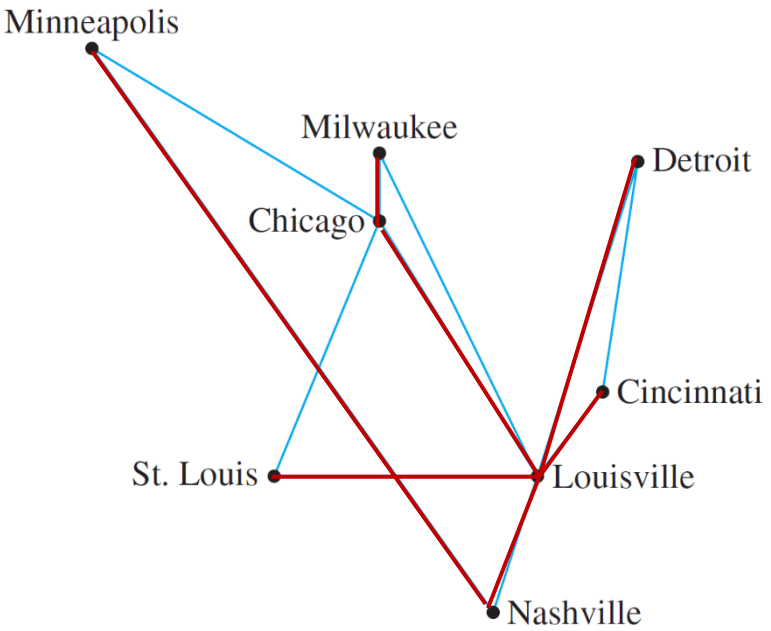


Figure 10.7.2

Lemma 10.5.3

If G is any connected graph, C is any circuit in G, and one of the edges of C is removed from G, then the graph that remains is still connected.

Spanning Trees

Definition: Spanning Tree

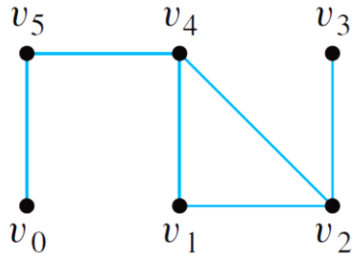
A **spanning tree** for a graph G is a subgraph of G that contains every vertex of G and is a tree.

Proposition 10.7.1

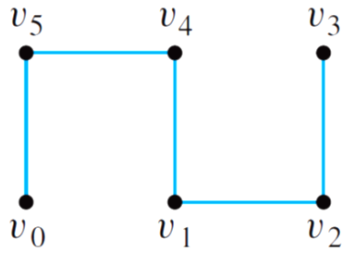
1. Every connected graph has a spanning tree.
2. Any two spanning trees for a graph have the same number of edges.

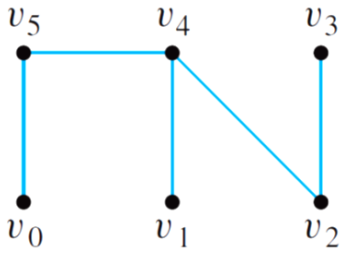
Example: Find all spanning trees for the graph G.

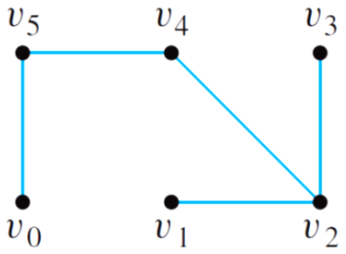
G



The graph G has one circuit $v_2v_1v_4v_2$ and removal of any edge of the circuit gives a tree. Hence there are 3 spanning trees for G.







Minimum Spanning Trees

The graph of the routes allowed by the Federal Aviation Authority shown in Figure 10.7.1 can be annotated by adding the distances (in miles) between each pair of cities. This is called a weighted graph.

Now, suppose the airline company wants to serve all the cities shown, but with a route system that minimizes the total mileage.

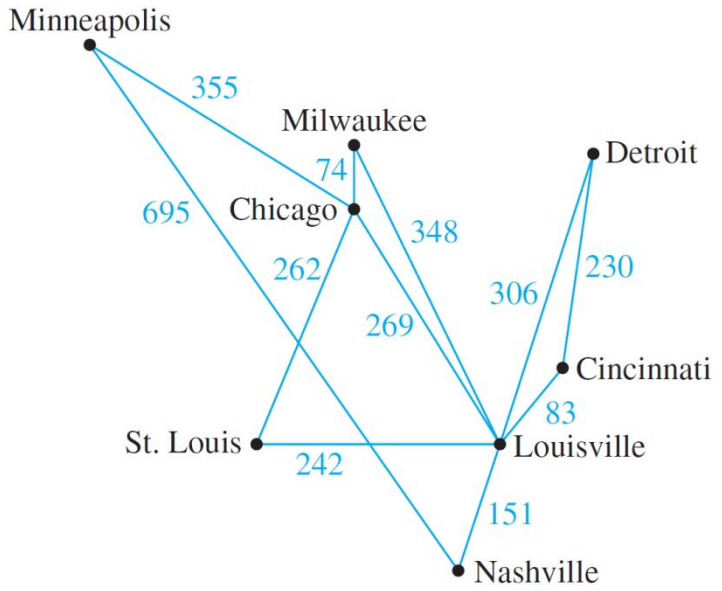


Figure 10.7.3

Minimum Spanning Trees

Definition: Weighted Graph

A **weighted graph** is a graph for which each edge has an associated positive real number **weight** . The sum of the weights of all the edges is the **total weight** of the graph.

Definition: Minimum Spanning Tree

A **minimum spanning tree** for a connected weighted graph is a spanning tree that has the least possible total weight compared to all other spanning trees for the graph.

If G is a weighted graph and e is an edge of G, then $\boldsymbol{w(e)}$ denotes the weight of e and $\boldsymbol{w(G)}$ denotes the total weight of G.

Minimum Spanning Trees: Kruskal's Algorithm



Joseph B. Kruskal
(1928 – 2010)

In Kruskal's algorithm (1956), the edges of a connected weighted graph are examined one by one in order of increasing weight.

At each stage the edge being examined is added to what will become the minimum spanning tree, provided that this addition does not create a circuit.

After $n - 1$ edges have been added (where n is the number of vertices of the graph), these edges, together with the vertices of the graph, form a minimum spanning tree for the graph.

Minimum Spanning Trees: Kruskal's Algorithm



Joseph B. Kruskal
(1928 – 2010)

Algorithm 10.7.1 Kruskal

Input: G [a connected weighted graph with n vertices]

Algorithm:

1. Initialize T to have all the vertices of G and no edges.
2. Let E be the set of all edges of G, and let $m = 0$.
3. While ($m < n - 1$)
   - 3a. Find an edge e in E of least weight.
   - 3b. Delete e from E.
   - 3c. If addition of e to the edge set of T does not produce a circuit, then add e to the edge set of T and set $m = m + 1$.

   End while

Output: T [T is a minimum spanning tree for G]

Minimum Spanning Trees: Kruskal's Algorithm

Example: Describe the action of Kruskal's algorithm on the graph shown in Figure 10.7.4, where $n = 8$.

	Edge considered	Wt	Action taken
1	→ Chi – Mil	74	added
2	→ Lou – Cin	83	added
3	→ Lou – Nas	151	added
4	→ Cin – Det	230	added
5	→ StL – Lou	242	added
6	→ StL – Chi	262	added
7	→ Chi – Lou	269	not added
8	→ Lou – Det	306	not added
9	→ Lou – Mil	348	not added
10	→ Min – Chi	355	added

Total weight = **1397**

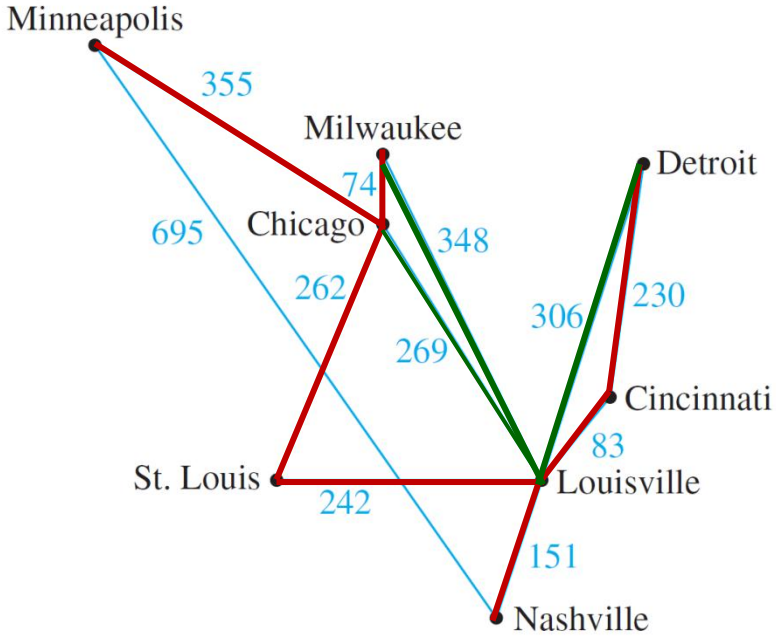


Figure 10.7.4



Joseph B. Kruskal
(1928 – 2010)

Minimum Spanning Trees: Prim's Algorithm



Robert C. Prim
(1921 - 2021)

Prim's algorithm (1957) works differently from Kruskal's. It builds a minimum spanning tree *T* by expanding outward in connected links from some vertex.

One edge and one vertex are added at each stage. The edge added is the one of least weight that connects the vertices already in *T* with those not in *T*, and the vertex is the endpoint of this edge that is not already in *T*.

Minimum Spanning Trees: Prim's Algorithm



Robert C. Prim
(1921 - 2021)

Algorithm 10.7.2 Prim

Input: G [a connected weighted graph with n vertices]

Algorithm:

1. Pick a vertex v of G and let T be the graph with this vertex only.
2. Let V be the set of all vertices of G except v.
3. For $i = 1$ to $n - 1$
   - 3a. Find an edge e of G such that (1) e connects T to one of the vertices in V, and (2) e has the least weight of all edges connecting T to a vertex in V. Let w be the endpoint of e that is in V.
   - 3b. Add e and w to the edge and vertex sets of T, and delete w from V.

Output: T [T is a minimum spanning tree for G]

Minimum Spanning Trees: Prim's Algorithm



Robert C. Prim
(1921 - 2021)

Example: Describe the action of Prim's algorithm on the graph shown in Figure 10.7.6, using the Minneapolis vertex as a starting point.

	Vertex added	Edge added	Weight
0	Minneapolis		
1	Chicago	Min – Chi	355
2	Milwaukee	Chi – Mil	74
3	St. Louis	Chi – StL	262
4	Louisville	StL – Lou	242
5	Cincinnati	Lou – Cin	83
6	Nashville	Lou – Nas	151
7	Detroit	Cin – Det	230

Total weight = **1397**

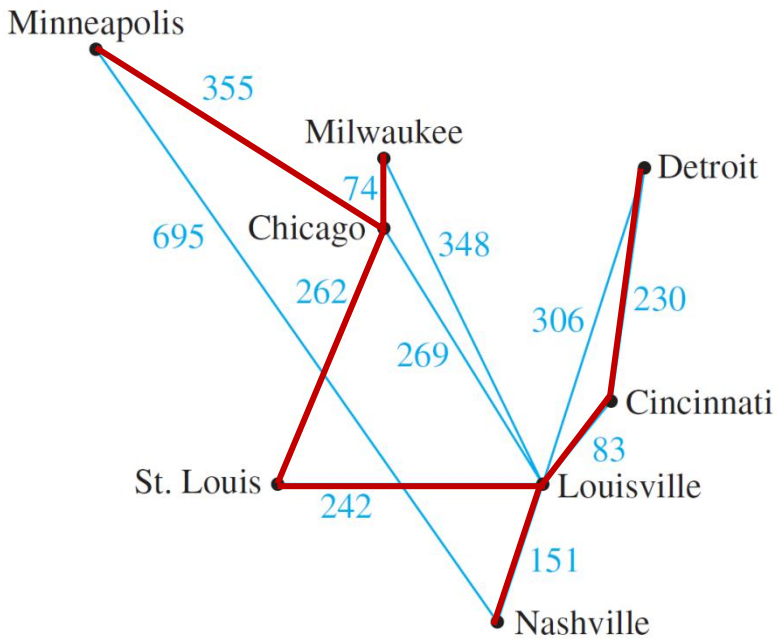


Figure 10.7.6

Tree Planting

15 December 2015

Former Dean
Prof David
Rosenblum



2 years on…

9 November 2017

十年树木，百年树人

It takes ten years to grow a tree, but a hundred years (a lifetime) to cultivate a person.



Tree Planting

9 November 2016

Dean

Prof Mohan Kankanhalli



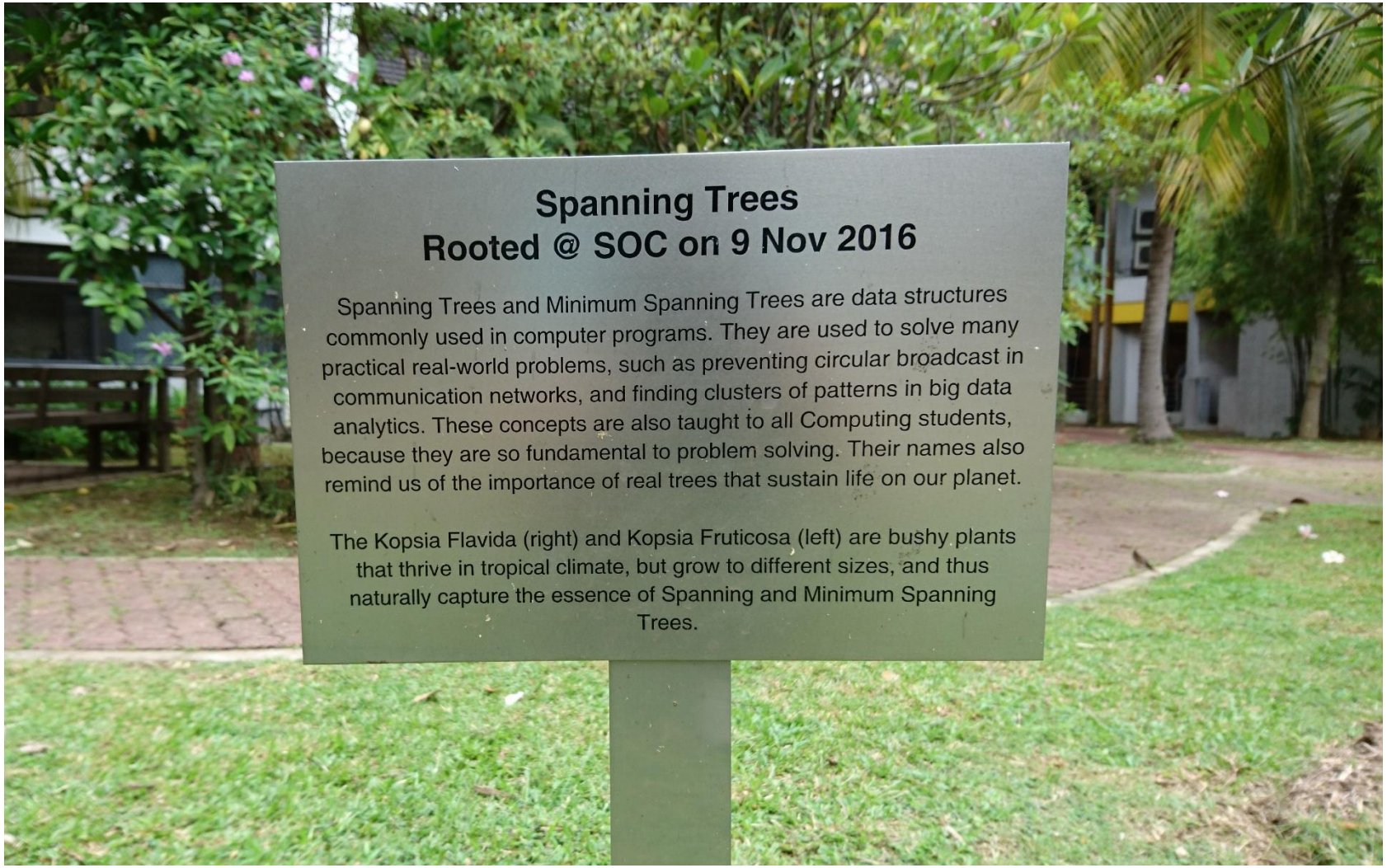
A year on…

9 November 2017



A year on…

9 November 2017



28 March 2024

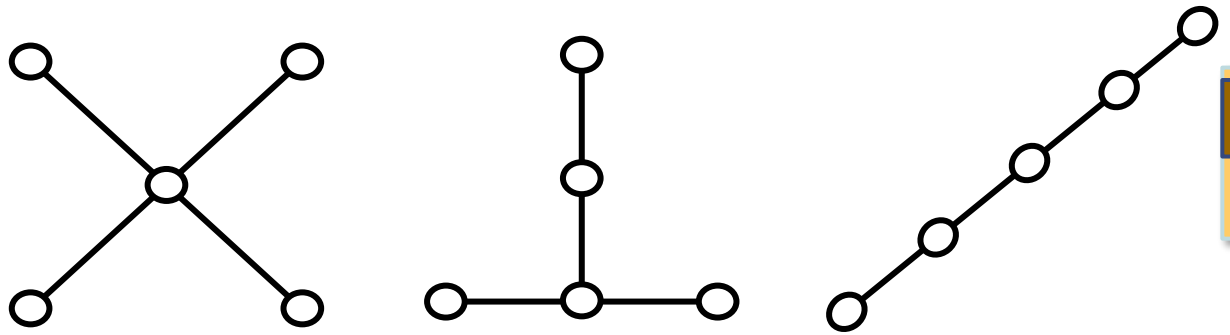




Exercise: Find all non-isomorphic trees with 5 vertices.



1. Since there are 4 edges in a tree with 5 vertices (Theorem 10.5.2), the total degree of the tree is $2\times4 = 8$ (Theorem 10.1.1).
2. A non-trivial tree has at least 2 vertices with degree one. The remaining 3 vertices, with a total degree of 6, have the following possible distributions of their individual degrees: $\{1,1,4\}$, $\{1,2,3\}$, or $\{2,2,2\}$.
3. Hence, there are **3** non-isomorphic trees with 5 vertices.



Theorem 10.5.2

Any tree with n vertices ($n > 0$) has $n - 1$ edges.

Theorem 10.1.1 The Handshake Theorem

Total degree of a graph $G = (V, E)$ is $2|E|$.

Past-Year's Exam Question!

AY2024/25 Semester 1 Q25

The **chromatic number** of an undirected graph G, denoted as $\chi(G)$, is the minimum number of colours required to colour the vertices of G in such a way that no two adjacent vertices have the same colour.

The **chromatic index** of an undirected graph G, denoted as $\chi'(G)$, is the minimum number of colours required to colour the edges of G in such a way that no two adjacent edges have the same colour.

A **cycle graph** (or circular graph) is an undirected connected graph that has at least 3 vertices and consists of a single cycle where every vertex is on the cycle. The cycle graph with n vertices is denoted as C_n.

Let $G = (V, E)$ be a connected undirected graph. A **vertex cut set** is a set S, where $S \subseteq V$, such that the removal of the vertices in S, along with their incident edges, disconnects G (that is, resulting in G having more than one connected component).
A **minimum vertex cut set** is a vertex cut set of the smallest possible size.

Past-Year's Exam Question!

AY2024/25 Semester 1 Q25

25(a) What is the chromatic number of each of the following graphs?

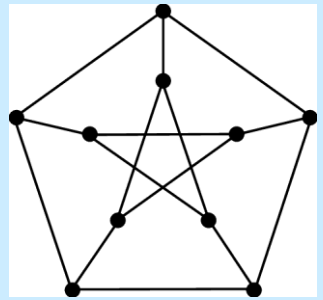
(i) K_{1231} 1231 $\chi(K_n) = n$

(ii) $K_{1231,1231}$ 2

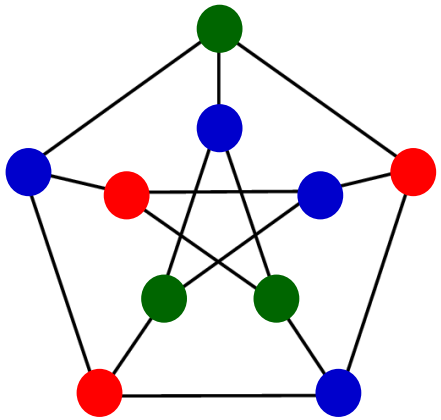
(iii) C_{1231} 3

$\chi(C_n) = 2$ if n is even,
$\chi(C_n) = 3$ if n is odd.

(iv) The Petersen graph as shown on the right. 3



Petersen graph



The **chromatic number** of an undirected graph G, denoted as $\chi(G)$, is the minimum number of colours required to colour the vertices of G in such a way that no two adjacent vertices have the same colour.

Past-Year's Exam Question!

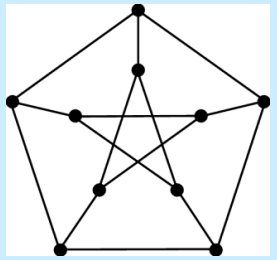
AY2024/25 Semester 1 Q25

25(b) What is the chromatic index of each of the following graphs?

(i) K_{100} 99

(ii) $K_{1231,1231}$ 1231

(iii) The Petersen graph as shown on the right. 4

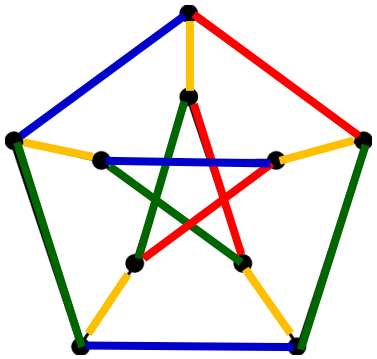


Petersen graph

Vizing's Theorem: $\chi'(K_n) = n$ if n is odd, $\chi'(K_n) = n - 1$ if n is even.
https://www.youtube.com/watch?v=v6OK5eSdOYk
https://www.youtube.com/watch?v=0F55ZA48xEE
https://askfilo.com/user-question-answers-algebra-2/find-the-edge-chromatic-number-of-kn-when-n-is-a-positive-35383434373733

Konig Theorem: If G is bipartite, then $\chi'(G) = \Delta(G)$, where $\Delta(G)$ is maximum degree of G.

Petersen graph is a snark, which is an undirected graph with exactly 3 edges per vertex whose edges cannot be coloured with only 3 colours. Snarks have chromatic index of 4 and certain connectivity requirements.

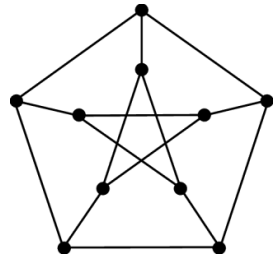


The **chromatic index** of an undirected graph G, denoted as $\chi'(G)$, is the minimum number of colours required to colour the edges of G in such a way that no two adjacent edges have the same colour.

Past-Year's Exam Question!

AY2024/25 Semester 1 Q25

25(c) Is the Petersen graph Eulerian? Is it Hamiltonian?



Petersen graph

The Petersen graph is neither Eulerian (all vertices have odd degrees) nor Hamiltonian.

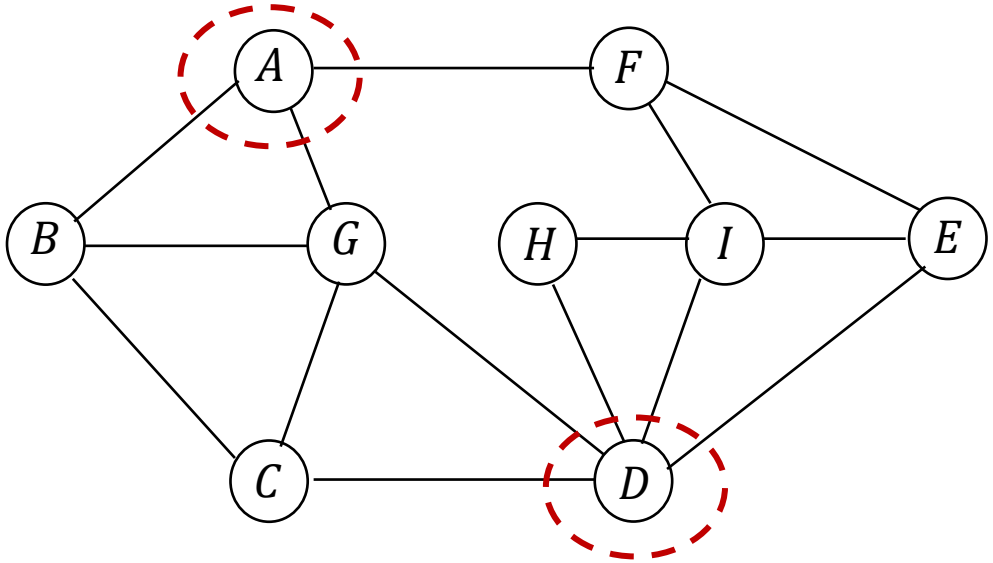
Note: The Petersen graph has a Hamiltonian path but not a Hamiltonian cycle.

https://ptwiddle.github.io/MAS341-Graph-Theory-2017/lecturenotes/lecture5.html

Past-Year's Exam Question!

AY2024/25 Semester 1 Q25

25(d) Write out a minimum vertex cut set for the graph below.



Minimum cut vertex set = $\{A, D\}$, $\{D, F\}$ or $\{D, I\}$.

Let $G = (V, E)$ be a connected undirected graph.
A **vertex cut set** is a set S, where $S \subseteq V$, such that the removal of the vertices in S, along with their incident edges, disconnects G (that is, resulting in G having more than one connected component).
A **minimum vertex cut set** is a vertex cut set of the smallest possible size.

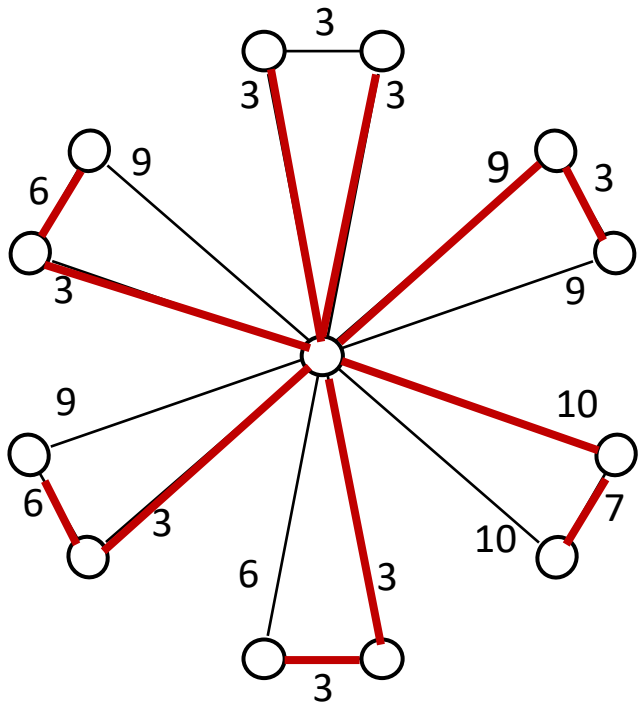
Past-Year's Exam Question!

AY2024/25 Semester 1 Q25

25(e) An undirected weighted graph is shown below.

(i) What is the weight of its minimum spanning tree?

(ii) How many minimum spanning trees are there if the vertices are labelled distinctly?



(i) 59

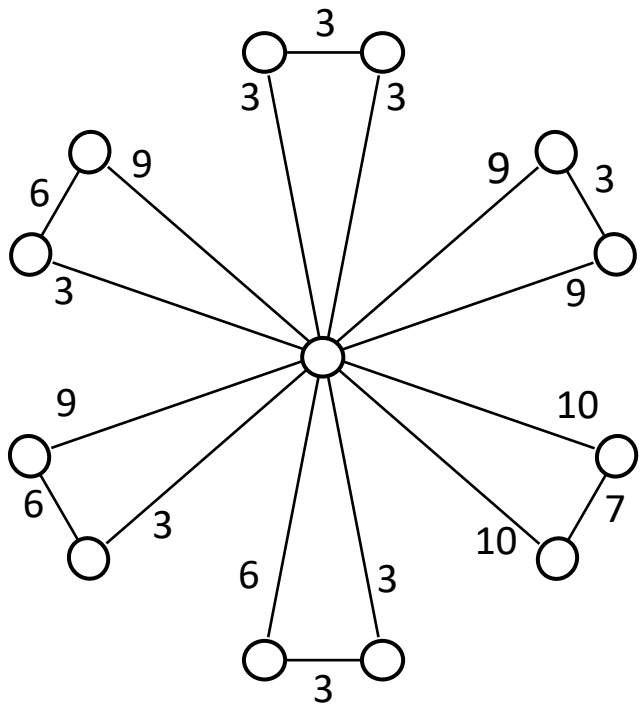
(ii) $3\times2\times2\times1\times1\times1 = \mathbf{12}$

Past-Year's Exam Question!

AY2024/25 Semester 1 Q25

25(f) (i) What is the name of the graph in part (e) above?

(ii) Given such a graph with $2n + 1$ vertices, what is its chromatic index for $n > 1$?



(i) Friendship graph (introduced in tutorial 10 Q7)

(ii) $2n$ for $n > 1$

The **chromatic index** of an undirected graph G, denoted as $\chi'(G)$, is the minimum number of colours required to colour the edges of G in such a way that no two adjacent edges have the same colour.

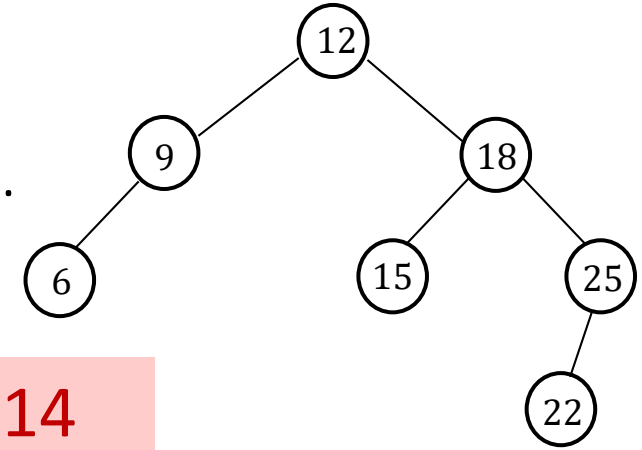
Past-Year's Exam Question!

AY2023/24 Semester 1

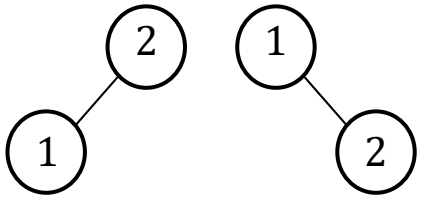
24f. A *Binary Search Tree* (BST) is a specialized form of a binary tree characterized by the following property: for any given vertex with the value x, all elements in its left subtree are smaller than x, and all elements in its right subtree are greater than x. You may assume that there are no duplicate values.

The diagram on the right shows an example of a BST with 7 values.

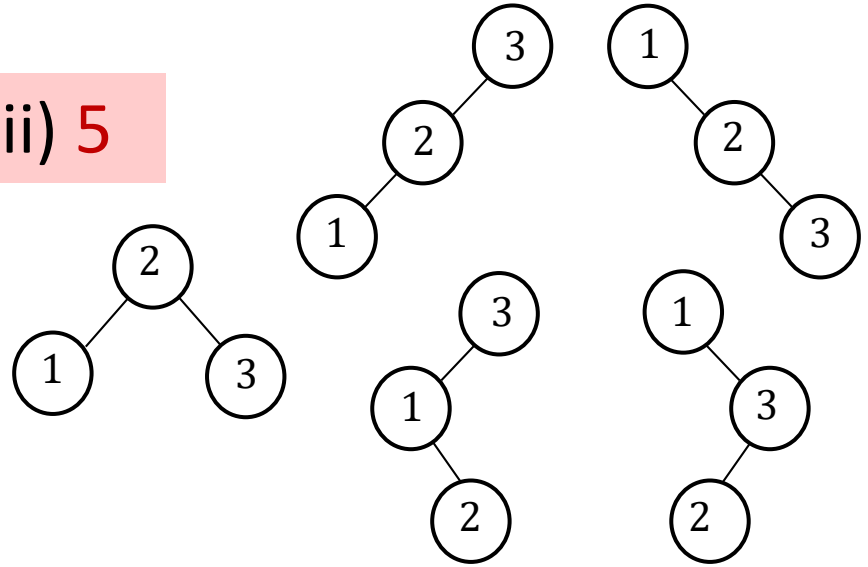
Find the number of distinct BSTs with (i) 2 values, (ii) 3 values, (iii) 4 values.



(i) 2



(ii) 5



(iii) 14

Catalan numbers

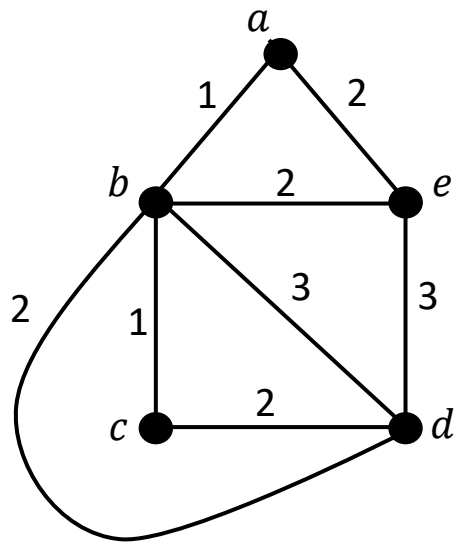
$$C_n = \frac{(2n)!}{(n+1)!\,n!}$$

$1,2,5,14,42,132,429,1430,4862, \ldots$

Past-Year's Exam Question!

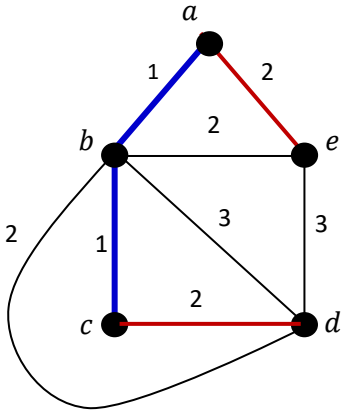
AY2020/21 Semester 2

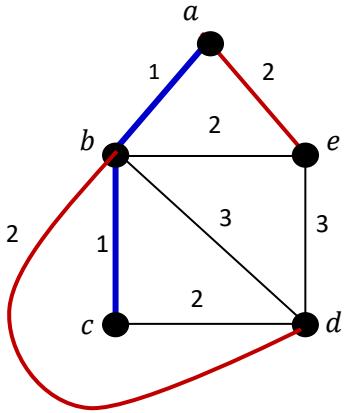
21b. In the following weighted graph, how many minimum spanning trees are there?

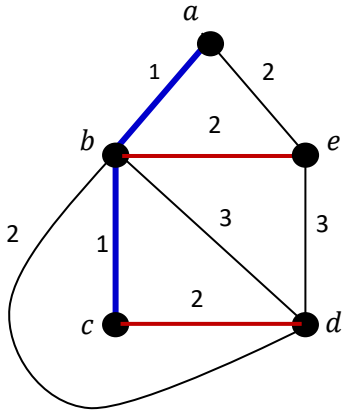


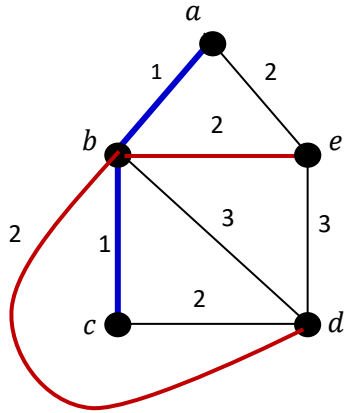
Answer: 4.

The MST consists of 4 edges, 2 of which have weight 1. The remaining 2 have weight 2. There are $\binom{4}{2} = 6$ choices, but 2 of them result in a circuit.









Past-Year's Exam Question!

AY2020/21 Semester 2

21e. Given two simple undirected graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ where $V_1 \cap V_2 = \emptyset$ and $E_1 \cap E_2 = \emptyset$, the *graph union*, *graph join* and *graph product* are defined as follows:

Graph union:

The union $G_\cup = G_1 \cup G_2$ has vertex set $V_\cup = V_1 \cup V_2$ and edge set $E_\cup = E_1 \cup E_2$.

Graph join:

The join $G_+ = G_1 + G_2$ has vertex set $V_+ = V_1 \cup V_2$ and edge set $E_+ = E_1 \cup E_2 \cup$ {all edges connecting every vertex in V_1 with every vertex in V_2}.

Graph product:

The product $G_\times = G_1 \times G_2$ has vertex set $V_\times = V_1 \times V_2$ (Cartesian product of V_1 and V_2) and two vertices (α, β), $(\gamma, \delta) \in V_\times$ are connected by an edge if and only if the vertices $\alpha, \beta, \gamma, \delta$ satisfy the following (with $\sim$ denoting "is adjacent to"):

$$(\alpha = \gamma \wedge \beta \sim \delta) \vee (\beta = \delta \wedge \alpha \sim \gamma)$$

Past-Year's Exam Question!

AY2020/21 Semester 2

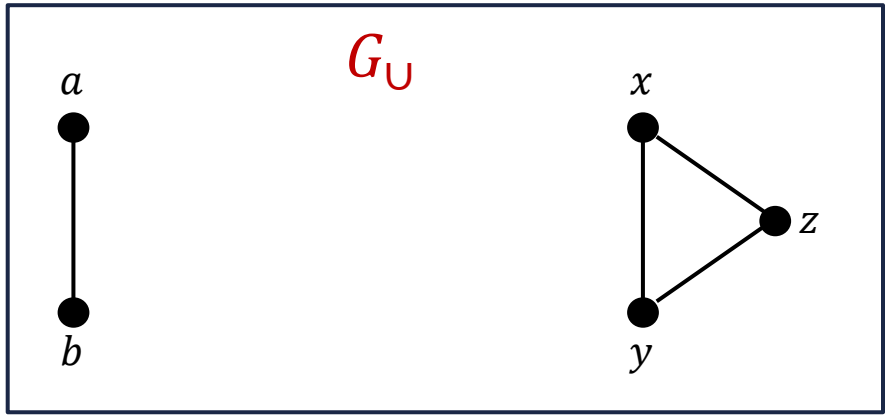
21e. Given two simple undirected graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ where $V_1 \cap V_2 = \emptyset$ and $E_1 \cap E_2 = \emptyset$, the *graph union*, *graph join* and *graph product* are defined as follows:

Graph union:

The union $G_\cup = G_1 \cup G_2$ has vertex set $V_\cup = V_1 \cup V_2$ and edge set $E_\cup = E_1 \cup E_2$.

Given the following graphs G_1 and G_2, draw their union graph.





Past-Year's Exam Question!

AY2020/21 Semester 2

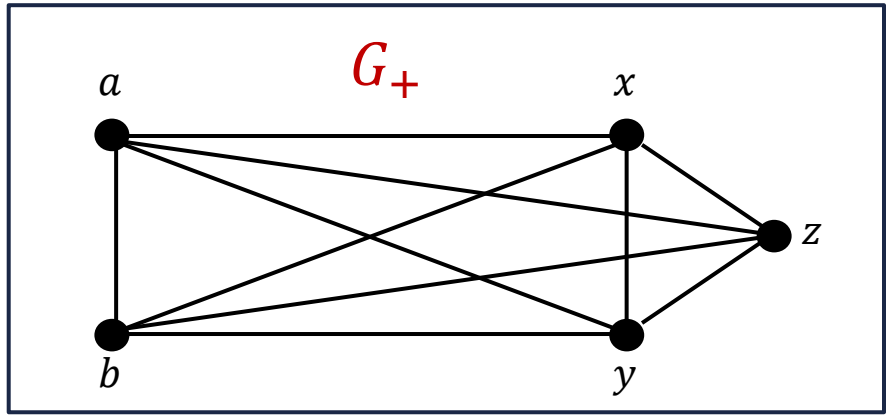
21e. Given two simple undirected graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ where $V_1 \cap V_2 = \emptyset$ and $E_1 \cap E_2 = \emptyset$, the *graph union*, *graph join* and *graph product* are defined as follows:

Graph join:

The join $G_+ = G_1 + G_2$ has vertex set $V_+ = V_1 \cup V_2$ and edge set $E_+ = E_1 \cup E_2 \cup$ {all edges connecting every vertex in V_1 with every vertex in V_2}.

Given the following graphs G_1 and G_2, draw their join graph.





Past-Year's Exam Question!

AY2020/21 Semester 2

21e. Given two simple undirected graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ where $V_1 \cap V_2 = \emptyset$ and $E_1 \cap E_2 = \emptyset$, the *graph union*, *graph join* and *graph product* are defined as follows:

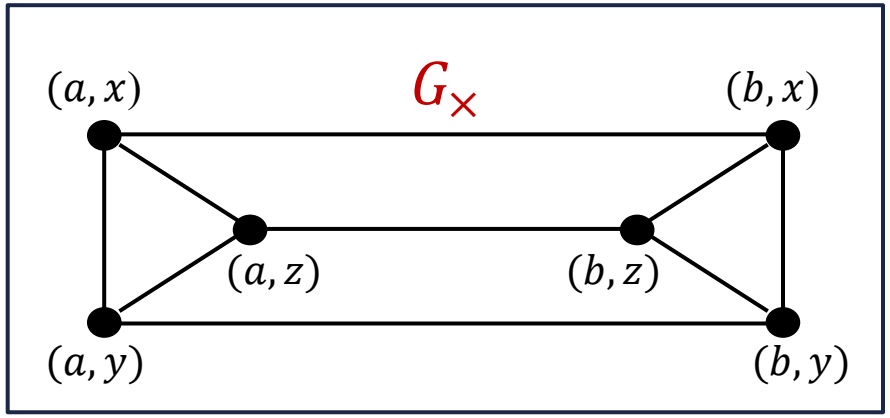
Graph product:

The product $G_\times = G_1 \times G_2$ has vertex set $V_\times = V_1 \times V_2$ (Cartesian product of V_1 and V_2) and two vertices $(\alpha, \beta), (\gamma, \delta) \in V_\times$ are connected by an edge if and only if the vertices $\alpha, \beta, \gamma, \delta$ satisfy the following (with $\sim$ denoting "is adjacent to"):

$$(\alpha = \gamma \ \wedge \ \beta \sim \delta) \vee (\beta = \delta \ \wedge \ \alpha \sim \gamma)$$

Given the following graphs G_1 and G_2, draw their product graph.





Recap #1: What You Have Learned in CS1231S

Mathematical induction

1. We have reached the end of the course.
2. By mathematical induction, if you survived the first lecture, and surviving one lecture implies you can survive the next, then congratulations — you have survived the entire course!
3. Unfortunately, surviving the course does not necessarily imply passing the exam. That requires a separate proof!

Recap #2: What You Have Learned in CS1231S

Sets and Cardinality

1. We began this course with sets, and now we have come to the end of the course.
2. The set of lectures remaining is empty.
3. The set of things you have learned is hopefully nonempty.
4. And the set of things you have yet to discover is infinite.

Recap #3: What You Have Learned in CS1231S

Proof by Contradiction

1. Suppose, for the sake of contradiction, that none of you learned anything from this course.
2. But you now know what ‘proof by contradiction’ means.
3. Therefore, we have a contradiction!
4. Hence, at least one thing has been learned. QED.

CS1231S: The Final Theorem

Theorem: All good things must come to an end.

Proof: This is our last lecture.

QED

Next week’s lectures

Make-up tutorial 11

Information on your final exam paper

END OF FILE